

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ №3
ТЕМА: Менеджер ресурсов и загрузка графических ассетов

ЦЕЛЬ

Разработать централизованную систему управления ресурсами (моделями, текстурами, шейдерами) с кэшированием, интегрировать загрузку 3D-моделей в игровой движок и обеспечить их визуализацию через компонент MeshRenderer на базе ECS.

ЗАДАЧИ

- 1. Подключить и настроить библиотеки для загрузки моделей (Assimp / tinyobjloader), изображений (stb_image) и работы с JSON (nlohmann/json).
- 2. Реализовать шаблонный класс ResourceManager с кэшированием загруженных ресурсов.
- 3. Создать загрузчики для конкретных типов ресурсов: Mesh, Texture, Shader.
- 4. Расширить RenderAdapter методами создания GPU-буферов, текстур и шейдерных программ.
- 5. Модифицировать компонент MeshRenderer для использования идентификаторов/ссылок на загруженные ресурсы.
- 6. Адаптировать RenderSystem для отрисовки объектов с загруженными мешами и текстурами.
- 7. Продемонстрировать загрузку и отображение нескольких 3D-моделей с текстурами.
- 8. (Бонус) Реализовать горячую замену шейдеров/конфигураций по изменению файла.

ГЛОССАРИЙ

Термин	Определение
Ресурс (Asset)	Данные, используемые движком: модели, текстуры, шейдеры, звуки, конфигурации.
ResourceManager	Центральный сервис, отвечающий за загрузку, кэширование и выдачу ресурсов.
Кэширование	Хранение загруженных ресурсов в памяти для повторного использования.
Mesh	Набор геометрических данных (вершины, индексы, атрибуты), готовый для отрисовки.
Texture	Изображение, загруженное в видеопамять и доступное в шейдерах.
Shader	Программа, выполняемая на GPU (вершинный, фрагментный, геометрический).
Сериализация	Процесс преобразования данных в формат для хранения/передачи (JSON, бинарный).

Горячая замена	Механизм перезагрузки ресурсов во время работы приложения без перезапуска.
----------------	--

НЕОБХОДИМЫЕ ИНСТРУМЕНТЫ

- **Язык:** C++17/20.
- **Среда разработки:** Visual Studio 2022 / CLion / VS Code.
- **Графический API:** OpenGL 3.3+ / DirectX 11 / Vulkan (через существующий RenderAdapter).
- **Математика:** GLM (рекомендуется) или собственная реализация.
- **Загрузка моделей:** Assimp (Open Asset Import Library) или tinyobjloader.
- **Загрузка изображений:** stb_image.h (однофайловая библиотека).
- **Работа с JSON:** nlohmann/json (рекомендуется) или RapidJSON.
- **Система сборки:** CMake.
- **Система контроля версий:** Git.

КРАТКАЯ ТЕОРЕТИЧЕСКАЯ СПРАВКА

Управление ресурсами — ключевой компонент любого игрового движка. Ресурсы (модели, текстуры, шейдеры) занимают значительный объём памяти и требуют длительной загрузки.

- **Проблема:** повторная загрузка одних и тех же данных с диска тратит время и память.
- **Решение:** централизованный менеджер с кэшем. Ресурс загружается однократно, все запросы получают указатель на существующий экземпляр.

Assimp — библиотека для импорта 3D-моделей в более чем 40 форматов. Она загружает сцену (aiScene), содержащую иерархию узлов, меши, материалы, текстуры.

STB Image — популярная однофайловая библиотека для загрузки изображений (PNG, JPEG, BMP и др.) в сырой массив пикселей.

Шейдеры — программы для GPU. В OpenGL они компилируются из исходных кодов (текстовых файлов). Кэширование скомпилированных программ ускоряет повторное использование.

Сериализация — сохранение/загрузка состояния сцены или настроек. В рамках ПЗ №3 сериализация может использоваться для описания набора ресурсов в JSON-файле (например, манифест ассетов).

Горячая замена — механизм, отслеживающий изменения файлов ресурсов на диске и автоматически перезагружающий их в рантайме. Позволяет итеративно разрабатывать шейдеры/текстуры без перезапуска движка.

ИСХОДНЫЕ ДАННЫЕ (с учётом предыдущих работ)

Работающий прототип движка, выполненный в рамках ПЗ №1 и ПЗ №2:

- Класс Application с игровым циклом и точным deltaTime.

- Абстрактный `RenderAdapter` и конкретная реализация для выбранного графического API.
- Базовая ECS-система: `World`, сущности, компоненты (`Transform`, `Tag`, `MeshRenderer`), `RenderSystem`.
- Возможность отрисовки нескольких геометрических примитивов (треугольник, квадрат) с различными трансформациями и цветами.

Все эти наработки должны быть расширены и адаптированы для работы с загружаемыми 3D-моделями и текстурами.

ЗАДАНИЕ И ПОРЯДОК ВЫПОЛНЕНИЯ

Шаг 1. Подключение внешних библиотек

- Добавить в проект `Assimp`, `stb_image`, `nlohmann/json`.
- Настроить линковку и пути включения (`CMake` или вручную).
- Написать простой тестовый код, проверяющий загрузку модели и текстуры (вывод информации в лог).

Результат: Проект успешно компилируется, тестовые функции загрузки выполняются без ошибок.

Шаг 2. Разработка базовой инфраструктуры `ResourceManager`

- Создать шаблонный класс `Resource<T>` (обёртка над загруженными данными, возможно с путём и счётчиком ссылок).
- Создать класс `ResourceManager` (рекомендуется синглтон или сервис-локатор).
- Реализовать метод `Load<T>(const std::string& path)` — если ресурс уже загружен, вернуть указатель из кэша; иначе вызвать соответствующий загрузчик, сохранить в кэше и вернуть.
- Кэш хранить как `std::unordered_map<std::string, std::shared_ptr<Resource<T>>>`.

Результат: Возможность загружать и повторно использовать ресурсы любого типа через единый интерфейс.

Шаг 3. Реализация загрузчика мешей (`MeshLoader`)

- Определить структуру `MeshData` (вершины, индексы, атрибуты: позиция, нормаль, UV).
- Используя `Assimp`, реализовать загрузку меша из файла (например, `.obj`, `.fbx`).
- Создать метод `ResourceManager::Load<Mesh>(path)`, который возвращает `std::shared_ptr<Mesh>`.
- Добавить в `RenderAdapter` метод `UploadMesh(const MeshData&)`, который создаёт вершинный/индексный буфер на GPU и возвращает хендл (или объект `GPUMesh`).
- Хранить GPU-хендл внутри `Mesh`.

Результат: Модель загружается с диска, преобразуется в формат для рендеринга, данные отправляются на GPU.

Шаг 4. Реализация загрузчика текстур (TextureLoader)

- Определить структуру TextureData (ширина, высота, каналы, массив пикселей).
- Используя stb_image, загрузить изображение.
- Реализовать ResourceManager::Load<Texture>(path).
- В RenderAdapter добавить CreateTexture(const TextureData&) — создаёт текстуру в видеопамяти, возвращает хендл.
- Хранить GPU-хендл внутри Texture.

Результат: Текстуры загружаются и кэшируются, готовы к использованию в материалах.

Шаг 5. Реализация загрузчика шейдеров (ShaderLoader)

- Определить структуру ShaderProgram (хендл GPU-программы, возможно также юниформы).
- Реализовать загрузку исходников вершинного и фрагментного шейдеров из текстовых файлов.
- Компиляция и линковка через вызовы графического API (инкапсулированные в RenderAdapter).
- Кэширование через ResourceManager.

Результат: Шейдеры загружаются и компилируются один раз, далее используются многократно.

Шаг 6. Модификация компонента MeshRenderer и RenderSystem

- Изменить компонент MeshRenderer: вместо PrimitiveType и Color хранить ссылки (или идентификаторы) на Mesh, Texture, Shader.
- Обновить RenderSystem: для каждой сущности с Transform и MeshRenderer:
 - получить ресурсы из ResourceManager;
 - применить матрицу трансформации;
 - установить шейдер и текстуру;
 - вызвать отрисовку меша через RenderAdapter.

Результат: Сущности ECS могут отображать произвольные загруженные 3D-модели с текстурами.

Шаг 7. Демонстрационная сцена

- Создать инициализационную сцену, в которой через ResourceManager загружаются:
 - модель "куб", "сфера" или любая готовая модель (например, teapot.obj, backpack.obj);

- текстура для модели (дерево, металл);
- базовый шейдер.
- Создать несколько сущностей с разными трансформациями, использующих одни и те же ресурсы.
- Запустить движок, убедиться в корректном отображении.

Результат: Рабочее приложение, демонстрирующее загрузку и отрисовку 3D-моделей с текстурами.

Шаг 8. (Бонус) Горячая замена конфигураций / шейдеров

- Реализовать фоновый поток или опрос файловой системы для отслеживания изменений файлов ресурсов.
- При обнаружении изменения перезагрузить ресурс и обновить кэш.
- Для шейдеров — перекомпилировать программу и заменить старую в момент использования.

Результат: Изменение шейдера или текстуры в файле приводит к немедленному обновлению на экране без перезапуска приложения.

ЗАДАНИЕ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. Сериализация сцены:

- Сохранить состояние сцены (сущности, их трансформации и ссылки на ресурсы) в JSON-файл.
- Реализовать загрузку сцены из JSON, автоматически подгружая необходимые ресурсы.

2. Асинхронная загрузка:

- Выполнять загрузку ресурсов в отдельном потоке.
- Пока ресурс не готов, отображать placeholder-модель.

3. Поддержка материалов:

- Создать компонент Material, хранящий параметры (цвета, шейдер, текстуры).
- MeshRenderer ссылается на Material как на отдельный ресурс.

4. Свой формат ресурсов:

- Разработать простой бинарный формат для хранения мешей/текстур, ускоряющий загрузку.

РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ РАБОТЫ

- **Управление памятью:** Используйте `std::shared_ptr` для ресурсов. Кэш хранит `shared_ptr`, а запросы возвращают его копию — ресурс остаётся в памяти, пока на него есть ссылки в компонентах.

- **Абстракция GPU:** Добавьте в `RenderAdapter` методы `CreateVertexBuffer`, `CreateIndexBuffer`, `CreateTexture2D`, `CompileShader`, `LinkProgram`. Это позволит легко сменить графический API.
- **Обработка ошибок:** При ошибке загрузки возвращайте `nullptr` или специальный "дефолтный" ресурс (например, белая текстура, сетка-заглушка).
- **Assimp:** Настройте импорт так, чтобы он автоматически триангулировал меши и генерировал нормали, если их нет.
- **Производительность:** Избегайте копирования больших массивов вершин — используйте перемещение или `emplace`.

ПОСЛЕДОВАТЕЛЬНОСТЬ ВЫПОЛНЕНИЯ

1. Подключение библиотек → проверка компиляции.
2. Реализация `Resource` и `ResourceManager`.
3. Загрузчик мешей + интеграция с GPU.
4. Загрузчик текстур + интеграция с GPU.
5. Загрузчик шейдеров + интеграция с GPU.
6. Модификация ECS-компонентов и системы рендеринга.
7. Демонстрационная сцена.
8. Бонус: горячая замена.

ФОРМА И СОДЕРЖАНИЕ ОТЧЕТА

1. **Титульный лист** (ФИО, группа, название работы).
2. **Введение** (цель, задачи).
3. **Архитектура решения:**
 - Диаграмма классов `ResourceManager`, `Resource`, загрузчиков.
 - Взаимодействие с `RenderAdapter` и ECS.
4. **Реализация ключевых модулей:**
 - Принцип работы кэша.
 - Фрагменты кода загрузки меша/текстуры/шейдера (с пояснениями).
 - Интеграция с `MeshRenderer`.
5. **Результаты:**
 - Скриншоты окна с загруженными 3D-моделями.
 - Скриншоты лога с информацией о загрузке.
6. **Трудности и их решение.**
7. **Заключение** (выводы, перспективы).

ПРИЛОЖЕНИЯ К ОТЧЕТУ

- Ссылка на Git-репозиторий с полным кодом проекта.
- CMakeLists.txt или файлы проекта.
- Пример JSON-манифеста ресурсов (если реализован).
- Видео-демонстрация работы (опционально).

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какие проблемы решает централизованный менеджер ресурсов?
2. В чём отличие между `unique_ptr` и `shared_ptr` для управления временем жизни ресурсов?
3. Почему кэш ресурсов часто реализуют как `unordered_map<string, weak_ptr<T>>`? Какие преимущества даёт `weak_ptr`?
4. Как Assimp представляет иерархию модели? Как получить отдельные меши и материалы?
5. Какие этапы включает в себя загрузка текстуры через `stb_image` и последующая выгрузка в GPU?
6. Для чего необходима компиляция шейдеров? Какие виды шейдеров обычно используются в 3D-графике?
7. Опишите механизм горячей замены шейдера. Какие сложности могут возникнуть при замене юниформов?
8. Как адаптировать систему ресурсов для поддержки асинхронной загрузки?

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. **Game Engine Architecture** – Jason Gregory (главы "Resource Management", "The Rendering Engine").
2. **OpenGL SuperBible** – Graham Sellers (работа с буферами, текстурами, шейдерами).
3. Документация Assimp – <http://assimp.org>
4. STB Image – <https://github.com/nothings/stb>
5. Nlohmann JSON – <https://json.nlohmann.me>
6. Статья "Hot Reloading in Game Engines" – Our Machinery (<https://ourmachinery.com/post/hot-reloading/>)